recent version if there's a newer version. You can add development-only dependencies with the option `--env development`.

To undo the pin, the command is `bin/importmap unpin`, or, I suppose, you could just remove it from the file.

All this is quite nice, and works really easily in development. There's no build step, so there's no lag between making a change and being able to test it in the browser.

There are a few limitations that haven't been completely addressed:

- There's kind of no point in using import maps if you need to compile your code, so import maps don't work well with TypeScript, or JSX, or Elm, or whatever.
- You are also on your own as far as using JavaScript features that are supported in browser, there's no Babel or esbuild to transpile newer features in terms of older ones.
- There are some npm packages that are set up in a non-standard way and which might not be easy to load in an import map. I'd expect this to be less and less of a problem as time goes on—the first package that I encountered in the wild where this was a problem (the graphing package Chartkick) has already addressed it.
- It is not completely clear to me what the story is for libraries that import both JavaScript and CSS, like Animate.css.
- More generally, there's no support for CSS manipulation. Rails provides the tailwindcss-rails and dartsass-rails gems, which provide standalone command lines for managing CSS. You'll likely still need a `Procfile` and a watch process for CSS, but that'll still be much less hassle than a full front-end build system.

Again, take a look at Appendix 1, *Framework Swap* to see more about how this